



COMPANY TECHNOLOGY DEVELOPMENT PROGRAM

Full-Stack Development Program

A One-Month Self-Study, Practical Delivery & AI-Assisted Learning Program



Backend



Frontend



AI-Assisted



Delivery

What We'll Cover

01

Program Objective

02

Program Duration

03

Target Technology Areas

04

Learning Approach

05

AI Session for Developers

06

Week 1 — Foundations

07

Week 2 — Backend Multi-Stack

08

Week 3 — Frontend Frameworks

09

Week 4 — Final Assignment

10

Expected Deliverables

11

Final Note to Participants



Program Objective

Build practical Enterprise Full-Stack capabilities across the main technologies used within the company.

The program focuses on self-study, hands-on practice, AI-assisted learning, and practical delivery. The goal is not only to learn syntax or frameworks, but to improve the ability to understand, develop, debug, and deliver small full-stack features across backend and frontend technologies. By the end of the month, each participant should have a clearer understanding of their current capability level, strengths, improvement areas, and readiness for future full-stack assignments.



Self-Study



Hands-On Practice



AI-Assisted Learning



Practical Delivery

Program Duration

1

Month
Total Duration

4

Weekly Learning
& Practice Phases

4

Self-Assessment
Checkpoints

1

Final Mini
Full-Stack Assignment



Each week combines self-study, practical exercises, and a self-assessment checkpoint — building progressively from engineering foundations to a full-stack final assignment.

Backend Technologies



PHP

Laravel

- Project structure
- Routes & Controllers
- Models & Validation
- ORM basics



Python

FastAPI / Django basics

- Routing
- Models / schemas
- Validation
- API development



.NET

ASP.NET Core / Entity Framework Core

- Controllers & Services
- Entity Framework Core
- Web API
- SQL Server integration



Node.js

Express / NestJS basics

- Routing & Middleware
- Async handling
- API development
- Error handling

Frontend Technologies



Angular

Component-based framework

- Components & Services
- Routing & Forms
- API calls
- Error handling & loading state



React JS

Component & hooks-based library

- Components & Hooks
- Routing & Forms
- State management
- API calls & loading state



Vue.js

Progressive frontend framework

- Components, Props & Events
- Routing & Forms
- API calls
- Error handling & loading state



TypeScript

Typed JavaScript foundations

- Types & Interfaces
- Async / await
- API models
- Type-safe development

Shared Engineering Skills

Common skills applied consistently across every backend and frontend stack.



REST APIs



SQL & Database Basics



Git & Pull Requests



Testing Basics



Debugging



Documentation



AI-Assisted Development

Self-Study, Supported by AI

This program is based mainly on self-study. A dedicated AI-focused session explains how developers can use AI tools effectively and safely throughout the learning and development process.

Participants Are Expected To

- 1 Follow the weekly learning plan
- 2 Use AI tools responsibly to support learning
- 3 Complete practical weekly exercises
- 4 Submit weekly self-assessment forms
- 5 Complete a final mini full-stack assignment
- 6 Be ready for a final technical evaluation

What the AI Session Covers

- How to use AI to understand a new technology stack
- How to ask AI to explain existing code
- How to generate learning plans
- How to create simple backend APIs with AI support
- How to create frontend components and forms
- How to generate test cases
- How to debug errors using AI
- How to review AI-generated code
- How to avoid unsafe AI usage

AI Usage Rules



Do not share secrets, credentials, tokens, or production data



Do not copy AI-generated code without understanding it



Do not use AI code in security, authentication, or payment logic without review



Always test the generated code



Always review and understand output before submitting



Document how AI was used during the assignment



Self-Study Topics

Objective: Build a common baseline across all participants before moving into multiple backend and frontend stacks.

- REST API concepts
- HTTP methods and status codes
- JSON request and response structure
- Authentication and authorization basics
- SQL basics
- Tables, relations, indexes, and migrations
- Git branching, commits, and pull requests
- Debugging using logs, stack traces, dev tools, and API tools
- Basic testing concepts
- Safe and effective AI usage for developers

Practical Exercise & Weekly Output



Practical Exercise

- A short summary of REST API basics
- A simple API request/response example
- A short Git workflow example
- Three useful AI prompts used for learning or code understanding



Weekly Output

- Week 1 self-assessment form
- Short learning notes
- AI prompt examples
- API basics exercise

Backend Stacks — Part I

PHP *Laravel*

- Laravel project structure
- Routes
- Controllers
- Models
- Validation
- ORM basics
- API response structure

Python *FastAPI / Django basics*

- FastAPI or Django basics
- Routing
- Models or schemas
- Validation
- API development
- Error handling

Backend Stacks — Part II



.NET

ASP.NET Core / Entity Framework Core

- ASP.NET Core project structure
- Controllers
- Services
- Entity Framework Core
- Web API
- SQL Server integration



Node.js

Express / NestJS basics

- Express or NestJS basics
- Routing
- Middleware
- Async handling
- API development
- Error handling

Practical Exercise & Weekly Output



Practical Exercise

Complete backend exercises in at least two backend stacks:

- Create a simple GET /items endpoint
- Create a simple POST /items endpoint
- Add basic validation
- Add basic error handling
- Use database interaction or mock data
- Prepare a short README explaining how to run it



Weekly Output

- Week 2 self-assessment form
- Backend exercise code or repository/branch
- Short comparison note between the backend stacks studied
- Main challenges faced during backend self-study

Frontend Frameworks — Part I

Angular

- Components
- Services
- Routing
- Forms
- API calls
- Error handling
- Loading state

React JS

- Components
- Hooks
- Routing
- Forms
- API calls
- Error handling
- State management
- Loading state

Frontend Frameworks — Part II



Vue.js

- Components
- Props
- Events
- Routing
- Forms
- API calls
- Error handling
- Loading state



TypeScript Basics

- Types
- Interfaces
- Async / await
- API models

Practical Exercise & Weekly Output



Practical Exercise

Build a simple frontend screen using Angular or Vue.js:

- List screen
- Create form
- API integration
- Loading state
- Error handling
- Basic validation messages



Weekly Output

- Week 3 self-assessment form
- Frontend exercise code or repository/branch
- Short note comparing Angular and Vue.js
- Main challenges faced during frontend self-study

Final Assignment Requirements

Objective: Evaluate the participant's ability to deliver a small feature end-to-end using backend, frontend, API integration, validation, and documentation.

- Backend endpoint
- Data model, database table, or mock persistence
- Frontend screen
- API integration
- Form validation
- Error handling
- Basic test or documented manual testing evidence
- README with setup and run steps
- AI usage notes
- Short demo or walkthrough

Suggested Mini Feature Ideas



Tasks Trello Clone



Employees Directory



Shortlinks Manager



Tickets Mini Module



Online Store



Cinema Booking Tickets



Notes Manager



Approval Workflow System

Weekly Output

The final evaluation checkpoint of the program — submitted at the end of Week 4.



Final assignment code or repository/branch



README file



Self-review checklist



AI usage log



Short demo or walkthrough



Week 4 self-assessment form

Expected Deliverables Dashboard



Week 1 Self-Assessment



Week 2 Self-Assessment



Week 3 Self-Assessment



Week 4 Self-Assessment



Weekly Learning Notes



Backend Exercises



Frontend Exercise



Final Mini Full-Stack Assignment



README / Setup Notes



AI Usage Log



Final Demo or Walkthrough

More Than a Training Exercise



This program is a baseline to help each developer identify their current level, strengths, gaps, and next development path.

The focus is on commitment, practical learning, problem solving, and the ability to use AI tools safely and effectively to accelerate learning and delivery.